Hadoop & Spark - ISEN 2025/2026

Lab: Data Quality

For this Lab you will need:

- The 3.5 Lab

As a reminder

Check 3.1 to login in NGROK, Kaggle, install Java or Spark and import libraries.

Handling corrupt data in a file

For this example of data corruption, please upload the 'RateCodes.**json**' (not csv) including in the data folder of this course.

```
from google.colab import files
files.upload()
```

You can drag and drop the file with this cell

Handling corrupt data in a file

If you open it you can see there is a format error on line 4:

```
{"RateCodeID":1,"RateCode":"Standard rate"}
{"RateCodeID":2,"RateCode":"JFK"}
{"RateCodeID":3,"RateCode":"Newark"}
{"RateCodeID":},"RateCode":"Westchester"} <- Here
{"RateCodeID":5,"RateCode":"Negotiated fare"}
{"RateCodeID":6,"RateCode":"Group ride"}
```

Handling corrupt data in a file

Let's try to read it normaly:

```
rateCodesJsonDF = (
                            spark
                              .read
                              .json("RateCodes.json")
                        )
rateCodesJsonDF.show(truncate=False)
```

We can see it adds a _corrupt_record column.

Handling corrupt data in a file

You can manually specify the option mode for handling errors, by default "PERMISSIVE" (see the doc here)

```
rateCodesJsonDF = (
                            spark
                              .read
                              .option("mode", "PERMISSIVE")
                              .json("RateCodes.json")
                          )
rateCodesJsonDF.show(truncate=False)
```

Handling corrupt data in a file

DROPMALFORMED:

```
rateCodesJsonDF = (
                            spark
                              .read
                              .option("mode", "DROPMALFORMED")
                              .json("RateCodes.json")
                          )
rateCodesJsonDF.show(truncate=False)
```

Handling corrupt data in a file

or FAILFAST:

```
rateCodesJsonDF = (
                            spark
                              .read
                              .option("mode", "FAILFAST")
                              .json("RateCodes.json")
                          )
rateCodesJsonDF.show(truncate=False)
```

Handling corrupt data in a file

Let's compare the PERMISSIVE option when reading a csv.

Upload 'RateCodes.**csv**' (not JSON):

```
from google.colab import files
files.upload()
```

Handling corrupt data in a file

Let's try it:

```
rateCodesCsvDF = (
                          spark
                            .read
                            .option("header", "true")
                            .option("mode", "PERMISSIVE")
                            .csv("RateCodes.csv")
                        )
rateCodesCsvDF.show(truncate=False)
```

Handling corrupt data in a file

Here is the result:

RateCodeID	RateCode	IsApproved
1	Standard Rate	Yes
2	JFK	Yes
3	Newark	Yes
4	null	null
Westchester	Yes	null
5	Negotiated fare	null
6	GroupRide	Yes

As you can see there is no `_corrupt_record`, but `null` values instead.

Handling corrupt data in a file

Extract from the PySpark doc:

PERMISSIVE: when it meets a corrupted record, puts the malformed string into a field configured by columnNameOfCorruptRecord, and sets malformed fields to null. To keep corrupt records, an user can set a string type field named columnNameOfCorruptRecord in an user-defined schema. If a schema does not have the field, it drops corrupt records during parsing. A record with less/more tokens than schema is not a corrupted record to CSV. When it meets a record having fewer tokens than the length of the schema, sets null to extra fields. When the record has more tokens than the length of the schema, it drops extra tokens.

Conclusion: you always have to read the documentation of a function to know the exact details of how things are handdled.

Cleaning

Lets take a new dataset:

https://www.kaggle.com/datasets/nelgiriyewithana/global-youtube-statistics-2023

```
!kaggle datasets download -d nelgiriyewithana/global-youtube-statistics-2023
!unzip global-youtube-statistics-2023.zip
```

Data quality

```
ytDF = spark.read
  .option("header","true")
  .option("delimiter",",")
  .option("inferShema","true")
  .csv('Global YouTube Statistics.csv')

ytDF.show(5)
```

Note: we should be specifying the schema manualy, but for simplcity sake we are just forcing 'created_year' and 'video views' as int:

```
from pyspark.sql.functions import *

ytDF = ytDF.withColumn("video views",col("video views").cast("integer"))
ytDF = ytDF.withColumn("created_year",col("created_year").cast("integer"))
ytDF = ytDF.withColumn("uploads",col("uploads").cast("integer"))
ytDF = ytDF.withColumn("highest_monthly_earnings",col("highest_monthly_earnings").cast("integer"))
ytDF.printSchema()
```

Data quality

Describe numerical values:

```
ytDF.describe("video views", "uploads", "created_year","highest_monthly_earnings")
```

Data quality

Filter (2 ways):

```
print("Before:" + str(ytDF.count()))

ytDF2 = (
    ytDF
      .where("created_year >= 2006")
      .filter(col("video views") > 1000)
      )

print("After:" + str(ytDF2.count()))

# Note: we do not save the result
```

Data quality

Replace values below 2006 with 2006:

```
print("Before:" + str(ytDF.count()))
ytDF3 = ytDF.withColumn("created_year", when((col("created_year") <= 2006), 2006).otherwise(col("created_year")))
print("After:" + str(ytDF3.count()))
```

```
ytDF3.describe("created_year")
```

Data quality

Drop rows with null values if any are present:

```
print("Before:" + str(ytDF3.count()))

ytDF4 = (
    ytDF3
      .na.drop('any')
      )

print("After:" + str(ytDF4.count()))
```

```
ytDF4.show(5)
```

Data quality

Fill wrong values with a mapping of default values:

```
ytDF3.describe("video views")
```

```
print("Before:" + str(ytDF3.count()))

defaultValues = {"video views":0}

ytDF5 = (
    ytDF3
      .na.fill(defaultValues)
      )

print("After:" + str(ytDF5.count()))
```

```
ytDF5.describe("video views")
```

Data quality

Drop duplicates rows if any:

```
print("Before:" + str(ytDF3.count()))

defaultValues = {"video views":0}

ytDF6 = (
    ytDF3
      .dropDuplicates(["Youtuber"])
      )

print("After:" + str(ytDF6.count()))
```

Exercice 1: Yellow Taxi

Lets apply cleaning on the Yellow Taxi Dataset on Kaggle:
https://www.kaggle.com/datasets/microize/newyork-yellow-taxi-trip-data-2020-2019

The goals with `yellow_tripdata_2020-01.csv`:

- 1 Creating the StructType object for the schema
- 2 Read the CSVs
- 3 Remove records outside the bounds : lpep_pickup_datetime [2020-01-03 : 2020-01-04[
- 4 Filter on passenger_count > 0 and trip_distance > 0
- 5 Set the default value of payment_type to 5 (Unknown) and RateCodeID to 1 (Standard rate)
- 6 Drop columns with nulls
- 7 Drop dupplicates rows

Bonus: apply it to all CSVs *(Tips: you can read multiple file by using .csv("yellow_tripdata*.csv"))*

Exercice 2: Open Exercice

Lets take a purposefuly messy and uncleaned dataframe :
https://www.kaggle.com/datasets/davidfuenteherraiz/messy-imdb-dataset

The goal:

- Read the file and declare its schema
- Clean the data

You can import it with:

```
!kaggle datasets download -d davidfuenteherraiz/messy-imdb-dataset
!unzip messy-imdb-dataset.zip
```